



WhatsApp AI Writing & Translation Assistant

Complete Technical Documentation & Step-by-Step Implementation Guide

[Node.js](#)[Express.js](#)[Meta WhatsApp Cloud API](#)[Groq Llama 3.3 70B](#)[Webhooks](#)

PROJECT OVERVIEW

When communicating in informal non-English languages or conversational Roman Urdu (e.g., "kysy ho", "muji joomna pasand h songs ky sath"), translating slang or regional context accurately into natural English can be challenging for standard engines.

This project bridges that gap by connecting **WhatsApp** directly to an ultra-fast **Open-Source AI Model (Llama 3.3 70B on Groq LPU)**. Whenever a user sends a text message to the WhatsApp bot, the system receives the message in real-time, interprets the linguistic context, translates or polishes the grammar, and sends back the final polished string in under **0.5 seconds**.



KEY FEATURES



Sub-Second AI Latency

Powered by Groq's custom LPU hardware to deliver instant translations without chat delays.



Roman Urdu & Urdu Translation

Accurately parses informal conversational slang and converts it into fluent English.



Grammar & Style Polishing

Enhances raw English text for professional, clean, and grammatically correct messaging.



Automated Webhook Processing

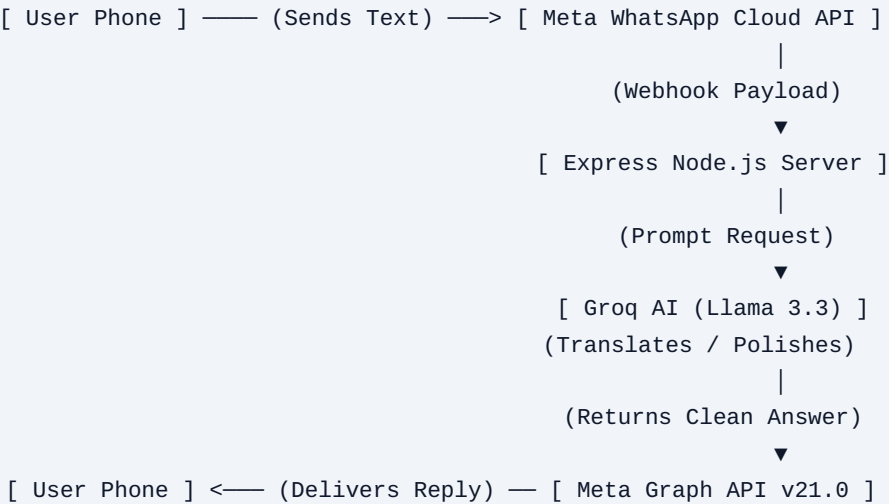
Receives incoming WhatsApp payloads instantly and handles outbound Graph API calls.



HOW IT WORKS (ARCHITECTURE FLOW)

The request flow follows a real-time event-driven webhook loop between WhatsApp servers, our local node server, and Groq's AI engine:

End-to-End Execution Sequence



TECH STACK & COMPONENT ROLES

Technology	Category	Role & Function in Project
Node.js & Express	Backend Server	Listens for HTTP requests on <code>/webhook</code> and coordinates API communications.
Meta Cloud API	Messaging Gateway	Official WhatsApp API that delivers incoming user messages and sends AI responses back.
Groq API (Llama 3.3)	AI Inference Engine	Processes Roman Urdu slang and translates text using high-speed hardware acceleration.
Axios	HTTP Client	Handles outgoing HTTP POST requests from Node.js to Meta Graph API endpoints.
ngrok	Secure Tunneling	Exposes local <code>localhost:3000</code> server to public HTTPS for Meta webhook validation.



STEP-BY-STEP INSTALLATION & SETUP

1. System Requirements

- **Node.js** (v18.0.0 or higher) installed on computer
- **VS Code** or any preferred IDE
- **ngrok** CLI installed for local tunnel forwarding

2. Dependencies Installation

```
# Clone project repository
git clone https://github.com/hannaessa16/whatsapp-ai-bot.git

# Navigate into working directory
cd whatsapp-ai-bot

# Install required Node.js libraries
npm install express axios
```

3. Configuration Variables (`index.js`)

Configure the credentials in your main script before launching:

```
const MY_VERIFY_TOKEN = "my_secret_bot_token_123"; // Custom secret token for Meta validation
const PHONE_NUMBER_ID = "YOUR_META_PHONE_NUMBER_ID";
const GRAPH_API_TOKEN = "YOUR_META_GRAPH_API_TOKEN";
const GROQ_API_KEY = "gsk_YOUR_GROQ_API_KEY";
```

4. Running the Project Locally

1. Start the Express server: `node index.js` (runs on port 3000).
2. In a separate terminal, start ngrok: `ngrok http 3000`.
3. Copy the generated `https://<subdomain>.ngrok-free.app` link.
4. In **Meta Developer Dashboard > WhatsApp > Configuration**, set Callback URL to:
`https://<subdomain>.ngrok-free.app/webhook`
5. Verify token and subscribe to the **messages** event field.

CORE CODE ARCHITECTURE (`INDEX.JS`)

The core application logic is split into three modular functions:

1. Webhook Handshake (GET /webhook)

Responds to Meta's automated challenge verification check during initial webhook configuration.

2. AI Inference Engine (getGroqResponse)

Constructs system prompts directing Llama 3.3 to handle Roman Urdu translation or English grammar polishing, returning clean, direct text.

3. Outbound Message Handler (sendWhatsAppMessage)

Constructs a POST payload to Meta Graph API `v21.0/<PHONE_NUMBER_ID>/messages` to send the AI response back to the user's phone.



KEY TROUBLESHOOTING & ERROR SOLUTIONS

Issue / Error	Root Cause	Resolution Applied
OAuth Code 190	Meta temporary developer token expired after 24 hours.	Refreshed token on Meta API Setup tab or generated a permanent System User Token.
Gemini Quota 429 Error	Strict API rate limits / model name deprecation.	Switched engine to Groq (Llama 3.3) offering 14,400 free daily requests.
Cannot GET /	Visiting root domain URL on browser preview.	Expected behavior—Express server only defines routes for <code>/webhook</code> endpoint.



Project Developer

Hanna Essa — Artificial Intelligence Student & Tech Strategist

PAF Institute of Applied Sciences and Technology